Quiero que me ayudes a generar la lógica fake del navegador de mi proyecto PBIO.

En este prompt quiero trabajar solamente con las funciones JavaScript que se comunican con el servidor REST.

Todavía no quiero hacer la interfaz.

La idea es:

UX
  ->
lógica fake
  ->
REST
  ->
lógica real
  ->
BBDD

La UX tiene que poder llamar a funciones normales sin preocuparse de fetch(), URLs ni códigos HTTP.

OBTENER DISPOSITIVOS

Quiero:

obtenerDispositivos()
-->
[Dispositivo]

Por dentro tiene que hacer:

GET /api/dispositivos

Tiene que recibir el JSON y devolver la lista de dispositivos.

No quiero que esta función rellene directamente un select ni toque HTML.

MEDICIONES DE UN DISPOSITIVO

Quiero:

uuid: Text
-->
obtenerMedicionesDispositivo()
-->
[Medicion]

Por dentro:

GET /api/dispositivos/{uuid}/mediciones

Tiene que devolver las mediciones de ese dispositivo.

Si el dispositivo existe pero no tiene datos:

[]

Si no existe:

quiero que devuelva un error que después pueda gestionar la UX.

Utiliza correctamente el UUID dentro de la URL.

ÚLTIMA MEDICIÓN

Quiero:

tipo: Text
-->
obtenerUltimaMedicion()
-->
Medicion

Por dentro tiene que llamar por ejemplo a:

GET /api/mediciones/ultima?tipo=CO2

y lo mismo para:

TEMPERATURA
RUIDO

Si no hay ninguna medición quiero que la UX pueda saberlo para después mostrar "Sin datos".

Pero no quiero que esta función escriba "Sin datos" directamente en la página.

TODAS LAS MEDICIONES

También quiero:

obtenerMediciones()
-->
[Medicion]

que utilice:

GET /api/mediciones

URL DEL SERVIDOR

Quiero la URL base en un solo sitio.

Por ejemplo:

API_BASE

No quiero repetir la dirección del servidor en todas las funciones.

FETCH Y ERRORES

Utiliza fetch().

Comprueba correctamente:

response.ok

códigos HTTP

errores de red

JSON incorrecto

servidor apagado

respuesta vacía

Quiero gestionar:

200
400
404
405
500

La lógica fake tiene que devolver esos errores a la parte de la interfaz.

No quiero alert() ni cambios de HTML desde estas funciones.

SEPARACIÓN

La lógica fake puede conocer:

- URLs;
- rutas REST;
- fetch();
- JSON;
- códigos HTTP.

Pero no quiero que se encargue de:

- crear tarjetas;
- rellenar tablas;
- cambiar un select;
- modificar HTML;
- CSS.

Eso lo haremos después en la UX.

Quiero poder hacer algo parecido a:

```
const dispositivos = await obtenerDispositivos();

const mediciones =
    await obtenerMedicionesDispositivo(uuid);
```

y que el resto de la web no necesite saber cómo se ha hecho la petición.

CÓDIGO DEL PROFESOR

Revisa primero el código web que ya nos dieron.

Había funciones que hacían de logicaFake mediante XMLHttpRequest.

Si sirven, puedes reutilizar la idea.

Pero revisa problemas como:

- parámetros de URL sin codificar;
- solamente comprobar status 200;
- no gestionar errores de red;
- JSON.parse sin control;
- URLs repetidas;
- lógica fake mezclada con interfaz.

Si hay cosas mal, corrígelas.

COMENTARIOS

Quiero comentarios en español.

No quiero comentarios obvios en cada línea.

Las funciones importantes tienen que llevar nuestra cabecera.

Ejemplo:

```
// obtenerDispositivos() --> [Dispositivo]
```

Otro:

```
// uuid: Text
// -->
// obtenerMedicionesDispositivo()
// -->
// [Medicion]
```

Otro:

// tipo: Text --> obtenerUltimaMedicion() --> Medicion

Usamos:

N = naturales
Z = enteros
R = reales
B = booleanos
Text = texto

[Tipo] = lista

Aunque en JavaScript utilices async/await o Promises, eso es un detalle de implementación y no hace falta ponerlo en la firma lógica.

TESTS

Quiero tests automáticos para esta capa.

No quiero tener que encender siempre el servidor real para probarla.

Utiliza fetch simulado o mocks.

Quiero probar como mínimo:

1. obtenerDispositivos() usa GET /api/dispositivos.

2. Procesar varios dispositivos.

3. Procesar lista vacía.

4. obtenerMedicionesDispositivo() utiliza la ruta correcta.

5. Comprobar el UUID en la URL.

6. Procesar varias mediciones.

7. Procesar lista vacía de mediciones.

8. obtenerUltimaMedicion("CO2").

9. obtenerUltimaMedicion("TEMPERATURA").

10. obtenerUltimaMedicion("RUIDO").

11. obtenerMediciones() usa GET /api/mediciones.

12. Simular 200.

13. Simular 400.

14. Simular 404.

15. Simular 500.

16. Simular servidor apagado.

17. Simular JSON incorrecto.

18. Comprobar que los errores no rompen toda la aplicación.

19. Comprobar que estas funciones pueden funcionar sin depender del HTML.

ANTES DE PROGRAMAR

Antes de cambiar nada:

1. Mira los archivos web del proyecto.
2. Busca la logicaFake que nos dio el profesor.
3. Dime qué se puede reutilizar.
4. Dime qué errores encuentras.
5. Dime qué archivos vas a crear o modificar.
6. Comprueba que no estás haciendo todavía la UX.
7. Comprueba que no estás metiendo aquí la lógica real del backend.

Después implementa y genera los tests.

Al terminar dime:

- qué archivos has creado;
- cuáles has modificado;
- qué funciones fake tienes;
- qué rutas REST utiliza cada función;
- cómo gestionas los errores;
- qué tests has creado;
- cuáles pasan;

- cuáles fallan;
- y qué problemas quedan.

No hagas BBDD, backend, Android, HTML ni CSS.

Quiero solamente la lógica fake del navegador.